

栈与队列的应用 实验报告

姓名：时分

学号：PB23151799

设计目标

在C语言环境下，利用栈与队列的相关操作，完成如下程序设计：

1. 简单的行文本编辑器

建立一个空的文本编辑器，输入指令条数N，从输入文件中读入N条操作指令并执行。操作指令包括：

1. Move k: 将光标移动到第 k 个字符之后，如果 k=0 则将光标移到文本第一个字符之前。
2. Insert n \n S: 在光标后插入长度为 n 的字符串 S，光标位置不变， $n \geq 1$ 。
3. Delete n: 删除光标后的 n 个字符，光标位置不变， $n \geq 1$ 。
4. Rotate n: 反转光标后的 n 个字符，光标位置不变， $n \geq 1$ 。
5. Get: 输出此时程序内保存的文本。
6. Prev: 光标前移一个字符。
7. Next: 光标后移一个字符。

对所有执行过的Get操作，将指定的内容写入输出文件。保证输入数据合法。

2. 括号配对检验

输入表达式数目N，输入N个含有英文字母、数字、四则运算符、左右大中小括号的表达式，以'@'作为表达式的结束符，检查表达式中的左右大中小括号是否匹配，若匹配则输出"YES"，否则输出"NO"。

3. 银行业务模拟

客户业务分为两种：第一种是申请从银行得到一笔资金，即取款或借款；第二种是向银行投入一笔资金，即存款或还款。银行有两个服务窗口，相应地有两个队列。客户到达银行后先排第一个队，处理每个客户业务时，如果属于第一种，且申请额超出银行现存资金总额而得不到满足，则立刻排入第二个队等候直至满足时才离开银行；否则业务处理完后立刻离开银行，每接待完一个第二种业务的客户，则顺序检查和处理(如果可能)第二个队列中的客户，对能满足的申请者予以满足，不能满足者重新排到第二个队列的队尾。任何时刻都只开一个窗口。假设检查不需要时间，营业时间结束时所有客户立即离开银行。设计上述银行业务的事件驱动模拟系统，通过模拟方法求出客户在银行内逗留的平均时间。

第一行输入四个数 N、total、close_time、average_time，分别表示来银行的总人数、银行开始营业时拥有的款额、今天预计的营业时长和客户交易时长，之后的 N 行每行输入两个数 a、b，第一个数 a 为客户办理的款额，用负值和正值分别表示第一类和第二类业务。第二个数 b 为客户来到银行的时间。

输出前 N 行分别对应每个客户的等候时长，第 N+1 行为客户等候平均时长（保留整数）。

简单的行文本编辑器

算法设计

采用两个顺序栈PText，NText分别存储光标前的文本和光标后的文本，相应功能用栈之间的元素移动实现。

数据结构

两个顺序栈共用字符数组V[MAXSIZE]的存储空间，PText栈底为V，NText栈底为V+MAXSIZE-1。
顺序栈类型定义如下：

```
typedef struct SqStack{
    char *base; //栈底
    char *top;  //栈顶
    int stacksize; //栈内元素个数
}SqStack;
```

程序结构

各函数原型、功能与接口描述如下：

```
int strcmp(char *A,char *B);
    //字符串比较函数，A>B返回1，A=B返回0，A<B返回-1
int Judgeope(char *ope);
    //判定输入的指令类型，返回指令序号
Status Move(SqStack &PText,SqStack &NText,int num);
    //Move指令，将一个栈的若干栈顶元素移动到另一个栈内，使PText栈内含有num个元素
Status Prev(SqStack &PText,SqStack &NText);
    //Prev指令，将PText栈顶元素出栈，放入NText
Status Next(SqStack &PText,SqStack &NText);
    //Next指令，将NText栈顶元素出栈，放入PText
Status Insert(SqStack &PText,SqStack &NText,FILE *fp,int num);
    //Insert指令，从文件fp读入num个字符存入PText，再将光标移动至原处
Status Get(SqStack &PText,SqStack &NText,FILE *nfp);
    //Get指令，输出当前内容至文件和显示器
Status Delete(SqStack &NText,int num);
    //Delete指令，将NText栈顶num个元素出栈
Status Rotate(SqStack &PText,SqStack &NText,int num);
    //Rotate指令，将NText栈顶num个元素倒序放入栈内
```

调试分析

测试数据Input.txt采用输入样例数据：

```
12
Insert 13
Balanced eert
Get
Move 2
Delete 5
Get
Next
Insert 6
editor
Move 0
Get
Move 10
Rotate 4
Get
```

发现存在如下问题：

1. Insert指令无法正常执行，少读入一个末尾字符。分析发现使用fgetc()函数时忽略了上一行的换行符。

解决方法：在读入需插入的字符前先通过fgetc()函数清除换行符。

2. Move指令光标移动距离比输入值短。检查发现移动字符时循环的结束条件中包括了每次移动都会改变的PText.stacksize。

解决方法：在函数开始时定义整型ssp标记PText.stacksize的初始值，循环结束条件中使用ssp。

3. Rotate指令执行后n个数据没有反转。检查发现放入时临时数组temp[]的循环方向为从尾到头。

解决方法：修改循环方向为从头到尾。

算法的时空分析

各函数时空复杂度如下：

```
int strcmp(char *A, char *B);
//字符串比较函数，A>B返回1，A=B返回0，A<B返回-1
//A,B中较短的长度为n
//时间复杂度：O(n)
//空间复杂度：O(1)
int JudgeOpe(char *ope);
//判定输入的指令类型，返回指令序号
//指令字符串长度为n
//时间复杂度：O(n)
//空间复杂度：O(1)
Status Move(SqStack &PText, SqStack &NText, int num);
//Move指令，将一个栈的若干栈顶元素移动到另一个栈内，使PText栈内含有num个元素
//时间复杂度：O(PText.stacksize+NText.stacksize)
//空间复杂度：O(1)
Status Prev(SqStack &PText, SqStack &NText);
//Prev指令，将PText栈顶元素出栈，放入NText
//时间复杂度：O(1)
//空间复杂度：O(1)
Status Next(SqStack &PText, SqStack &NText);
//Next指令，将NText栈顶元素出栈，放入PText
//时间复杂度：O(1)
//空间复杂度：O(1)
Status Insert(SqStack &PText, SqStack &NText, FILE *fp, int num);
//Insert指令，从文件fp读入num个字符存入PText，再将光标移动至原处
//时间复杂度：O(num)
//空间复杂度：O(num)
Status Get(SqStack &PText, SqStack &NText, FILE *nfp);
//Get指令，输出当前内容至文件和显示器
//时间复杂度：O(PText.stacksize+NText.stacksize)
//空间复杂度：O(1)
Status Delete(SqStack &NText, int num);
//Delete指令，将NText栈顶num个元素出栈
//时间复杂度：O(num)
//空间复杂度：O(1)
Status Rotate(SqStack &PText, SqStack &NText, int num);
//Rotate指令，将NText栈顶num个元素倒序放入栈内
//时间复杂度：O(num)
//空间复杂度：O(num)
```

测试结果及分析

文件Input.txt内容：

```
12
Insert 13
Balanced eert
Get
Move 2
Delete 5
Get
Next
Insert 6
editor
Move 0
Get
Move 10
Rotate 4
Get
```

显示器输出：

```
Balanced eert //完成第一次插入
Bad eert //完成光标移动与删除
Badeditor eert //完成第二次插入
Badeditor tree //完成光标移动与倒置
```

```
-----
Process exited after 0.08362 seconds with return value 1
请按任意键继续. . .
```

文件Output.txt内容：

```
Badeditor tree
```

括号配对检验

算法设计

读取表达式，遇到左括号则判断其优先级是否低于栈顶左括号的优先级（'(' < '[' < '{'）；如果是，则将该括号入栈；否则认为括号配对错误。遇到右括号则判断其是否与栈顶左括号匹配，若匹配则将左括号出栈；否则认为括号配对错误。在字符串读取完毕后，判断栈是否为空，若不为空则认为括号配对错误，否则括号配对正确。

数据结构

字符数组text[MAXSIZE]存储读入的表达式，整型数组result[num]存储各表达式的判断结果，顺序栈S存储读入的左括号。顺序栈类型定义如下：

```
typedef struct SqStack{
    char *base; //栈底
    char *top;  //栈顶
    int stacksize; //栈内元素个数
}SqStack;
```

程序结构

各函数原型、功能与接口描述如下：

```
Status InitStack(SqStack S);
    //构造空顺序栈S
Status Push(SqStack S, char c);
    //c入栈
char GetTop(SqStack S);
    //取栈顶元素
char Pop(SqStack S);
    //栈顶元素出栈
Status BracketCmp(char a, char b);
    //判定括号a是否比括号b优先级低，是返回TRUE，否则返回FALSE
Status BracketPairing(char a, char b);
    //判定右括号a是否与左括号b配对，是返回TRUE，否则返回FALSE
Status CheckBracket(char *text, SqStack S);
    //检查左/右括号是否满足要求，若满足返回TRUE，否则FALSE
```

调试分析

测试数据采用输入样例数据：

```
3
2*[(x+y)/(1-x)]@
(25+x)*(a*(a+b+b))}@
{1+2[c-d(7/9)]}@
```

发现存在如下问题：

1. 第一个左括号无法正常入栈。
解决方法：将入栈条件修改为：栈空或栈顶括号优先级高。
2. 右括号多于左括号时无法正常判断。
解决方法：将出栈条件修改为：栈不空且左右括号匹配。

算法的时空分析

各函数时空复杂度如下：

```
Status InitStack(SqStack S);
    //构造空顺序栈S
    //时间复杂度：O(1)
    //空间复杂度：O(n)
Status Push(SqStack S, char c);
    //c入栈
    //时间复杂度：O(1)
    //空间复杂度：O(1)
char GetTop(SqStack S);
```

```

//取栈顶元素
//时间复杂度:  $O(1)$ 
//空间复杂度:  $O(1)$ 
char Pop(SqStack S);
//栈顶元素出栈
//时间复杂度:  $O(1)$ 
//空间复杂度:  $O(1)$ 
Status BracketCmp(char a, char b);
//判定括号a是否比括号b优先级低
//时间复杂度:  $O(1)$ 
//空间复杂度:  $O(1)$ 
Status BracketPairing(char a, char b);
//判定右括号a是否与左括号b配对
//时间复杂度:  $O(1)$ 
//空间复杂度:  $O(1)$ 
Status CheckBracket(char *text, SqStack S);
//检查左/右括号是否满足要求, 若满足返回TRUE, 否则FALSE
//时间复杂度:  $O(n)$ 
//空间复杂度:  $O(1)$ 

```

测试结果及分析

键盘输入:

```

3
2*[(x+y)/(1-x)]@
(25+x)*(a*(a+b+b))@
{1+2[c-d(7/9)]@

```

显示器输出:

```

YES
NO
NO

```

银行业务模拟

算法设计

读取客户信息建立客户链表customers, 建立事件链表, 并将第一个客户的到达事件插入链表。直到事件链表空为止循环处理链表的第一个事件。全部处理完毕后, 计算仍未得到处理的客户的等待时间(即一直等待到银行关门)。

处理客户到达事件时, 将当前客户插入队列Q1, 将下一个客户到达事件插入事件表; 若队列Q1只剩一个客户, 将该客户离开事件插入事件表。

处理客户离开事件时, 移除队列Q1队头客户, 判断该用户是否去队列Q2; 再判断该用户是否是存款客户, 如果是则检查Q2; 若队列Q1不空, 队列Q1中下一个客户的离开事件插入事件表。

数据结构

采取事件驱动模拟。用链表存储各客户的信息(存/取款、到达时间、离开时间), 用链表存储各事件的信息。二者结点结构定义如下:

```

typedef struct Customer{
    int Money;
    int ArriveTime;
    int LeaveTime;
    struct Customer *next;
}Customer;

typedef struct Event{
    int OccurTime; //事件的发生时间
    int EventType; //事件类型, 1表示进入, 0表示离开
    struct Event *next;
}Event,*LinkList;

```

用两个队列Q1、Q2表示两个窗口的队伍，队列结构定义如下：

```

typedef struct QNode{
    Customer *ctm;
    struct QNode *next;
}QNode;

typedef struct Queue{
    QNode *front;
    QNode *rear;
}Queue,*LinkQueue;

```

全局变量N,total,close_time,average_time存储来银行的总人数、银行开始营业时拥有的款额、今天预计的营业时长和客户交易时长。

程序结构

各函数原型、功能与接口描述如下：

```

Status InputCustomer(Customer *C,int N);
    //将客户信息输入带头结点链表
Status EnQueue(LinkQueue Q, Customer *c);
    //将客户c插入队尾
Status EnList(LinkList events,int time,int EventType);
    //按时间顺序插入新事件
int QueueLength(LinkQueue Q);
    //检查队列Q长度
Customer *DeIFrontQueue(LinkQueue Q);
    //删去Q队头, 返回队头客户指针
Event DelFirstEvent(LinkList events);
    //删除事件列表第一个结点, 返回其事件
Status CustomerArrived(LinkList events,LinkQueue Q1,Customer *nctm);
    //客户到达事件
int CheckQ2(LinkQueue Q2,int BFTotal,int time);
    //检查Q2, 处理可以取款的客户, 返回Q2客户的个数
Status CustomerDeparture(LinkList events,Event pe,LinkQueue Q1,LinkQueue Q2);
    //客户离开事件

```

调试分析

测试数据采用输入样例数据：

```
4 10000 600 10
-2000 0
-11000 10
-10000 30
2000 50
```

发现存在如下问题：

1. 不能输出一一直在等待的客户的等待时间。

解决方法：在最后加入判断，若客户未离开则计算其等待时间。

2. 检查Q2时花费了多余时间检查。

解决方法：检查Q2时若银行资金总额小于初始值则停止循环。

算法的时空分析

各函数时空复杂度如下：

```
Status InputCustomer(Customer *C,int N);
    //将客户信息输入带头结点链表
    //时间复杂度:  $O(N)$ 
    //空间复杂度:  $O(N)$ 
Status EnQueue(LinkQueue Q, Customer *c);
    //将客户c插入队尾
    //时间复杂度:  $O(1)$ 
    //空间复杂度:  $O(1)$ 
Status EnList(LinkList events,int time,int EventType);
    //按时间顺序插入新事件
    //时间复杂度:  $O(\text{events.length})$ 
    //空间复杂度:  $O(1)$ 
int QueueLength(LinkQueue Q);
    //检查队列Q长度
    //时间复杂度:  $O(Q.length)$ 
    //空间复杂度:  $O(1)$ 
Customer *DeIFrontQueue(LinkQueue Q);
    //删去Q队头，返回队头客户指针
    //时间复杂度:  $O(1)$ 
    //空间复杂度:  $O(1)$ 
Event DeIFirstEvent(LinkList events);
    //删除事件列表第一个结点，返回其事件
    //时间复杂度:  $O(1)$ 
    //空间复杂度:  $O(1)$ 
Status CustomerArrived(LinkList events,LinkQueue Q1,Customer *nctm);
    //客户到达事件
    //时间复杂度:  $O(\text{events.length})$ 
    //空间复杂度:  $O(1)$ 
int checkQ2(LinkQueue Q2,int BFTotal,int time);
    //检查Q2，处理可以取款的客户，返回Q2客户的个数
    //时间复杂度:  $O(Q2.length)$ 
    //空间复杂度:  $O(1)$ 
Status CustomerDeparture(LinkList events,Event pe,LinkQueue Q1,LinkQueue
Q2);
    //客户离开事件
    //时间复杂度:  $O(\text{event.length})$ 
    //空间复杂度:  $O(1)$ 
```


测试结果及分析

键盘输入：

```
4 10000 600 10
-2000 0
-11000 10
-10000 30
2000 50
```

显示器输出：

```
0
590
30
0
155
```

实验体会与收获

通过本实验，我进一步了解了栈的后进先出、队列的先进先出的特点，掌握了栈、队列的表示和实现技术，掌握如何运用栈、队列的特点来构建算法，了解了事件驱动模拟的算法构建。